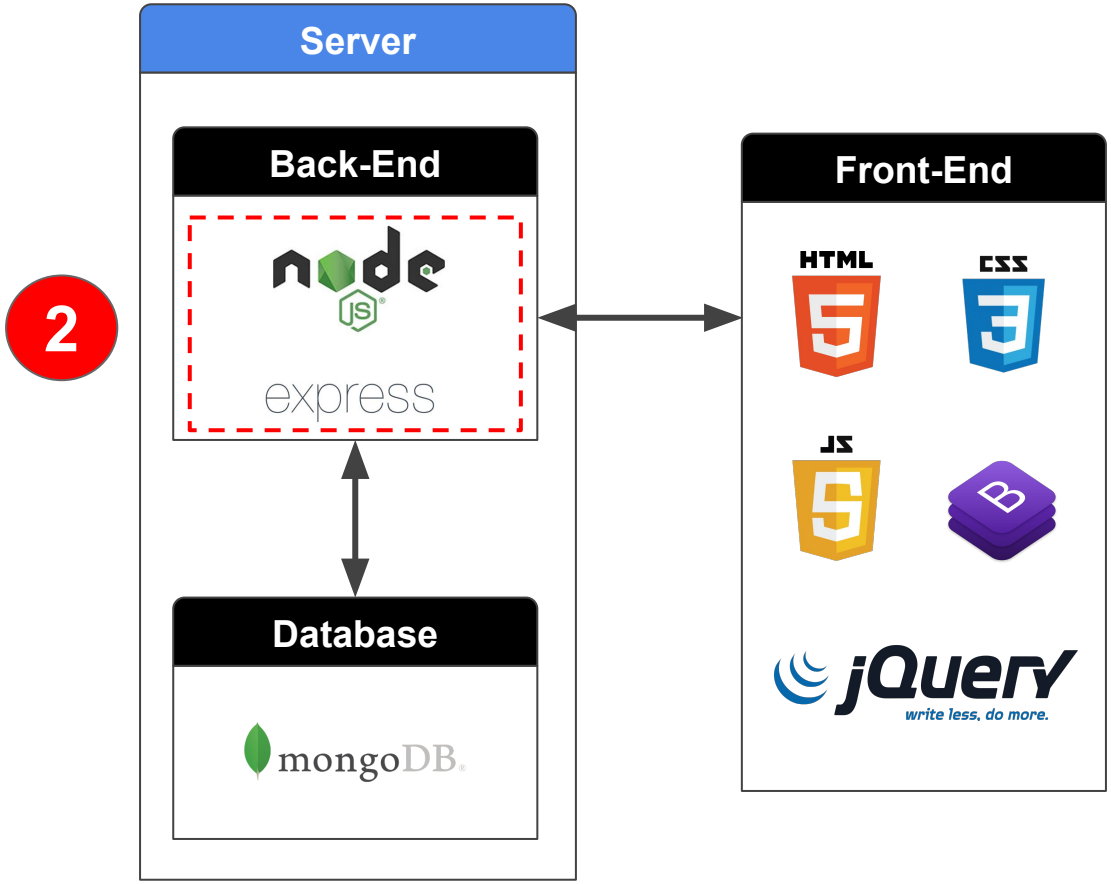
Node.js + Express

คือ Server Side Framework ยอดนิยม

+ express


2
Server
Back-End
node
express
Database
mongoDB
Front-End
HTML
CSS
JS
jQuery
write less, do more.

Express

Express

Home Getting started Guide API reference Advanced topics Resources

Express 4.16.1

Fast, unopinionated, minimalist web framework for Node.js

```
$ npm install express --save
```

Express 4.16.0 contains important security updates.

For more information on what was added in this release, see the 4.16.0 changelog.

Web Applications

Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

APIs

With a myriad of HTTP utility methods and middleware at your disposal, creating a robust API is quick and easy.

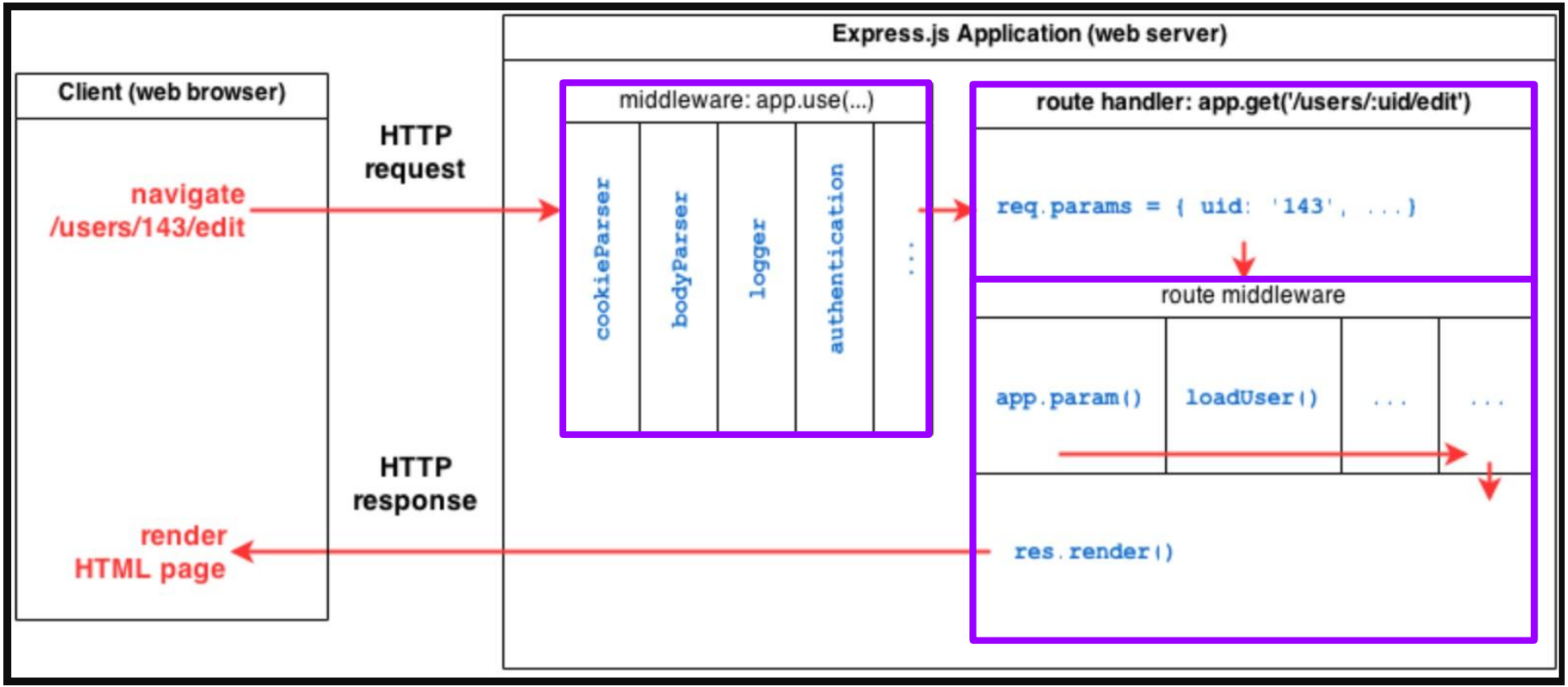
Performance

Express provides a thin layer of fundamental web application features, without obscuring Node.js features that you know and love.

Frameworks

Many popular frameworks are based on Express.

Express Structure



ติดตั้ง Express

```
npm install express --save          # ติดตั้ง express ภายใน Project บันทึกลง package.json
```

ติดตั้ง nodemon

```
npm install -g nodemon              # ติดตั้ง nodemon แบบ Global ทำให้เรียกคำสั่ง nodemon ได้
```

nodemon ช่วย Restart ตัว Application ที่กำลังทำงานอยู่ให้อัตโนมัติเมื่อมีการแก้ไขโค้ด ช่วยลดเวลาไม่ต้องทำการสั่งให้ Node.js ทำการ Run Application ใหม่ทุกครั้ง

Routing

Routing

Routing เป็นการกำหนด URI (Endpoints) และ Methods ภายใน Applications ว่าให้ตอบสนองอย่างไรเมื่อ Client ทำการเรียกเข้ามา

```
var express = require("express");
var app = express();

app.get("/", (req, res) => {
    res.send("hello world");
});

app.get("/users/:userId", (req, res) => {
   res.send(req.params);
});

app.listen(3000, () => {
    console.log("Server listening on port 3000!" );
});
```

http://localhost:3000

http://localhost:3000/users/olan
http://localhost:3000/users/jame

HTTP Methods

Methods	Task	Example
GET	Fetch all or any resource	**GET** /user - Fetch all user **GET** /user/olan - Fetch user olan
POST	Create a resource	**POST** /user - Create user olan data name=Olan&age=33
PUT	Update a resource	**PUT** /user/olan - Update user olan data name=Olan&age=33
DELETE	Delete a resource	**DELETE** /user/olan - Delete user olan

Route Methods

```
// GET method route
app.get("/", function(req, res) {
    res.send("GET request to the homepage");
});

// POST method route
app.post("/", function(req, res) {
    res.send("POST request to the homepage");
});
```

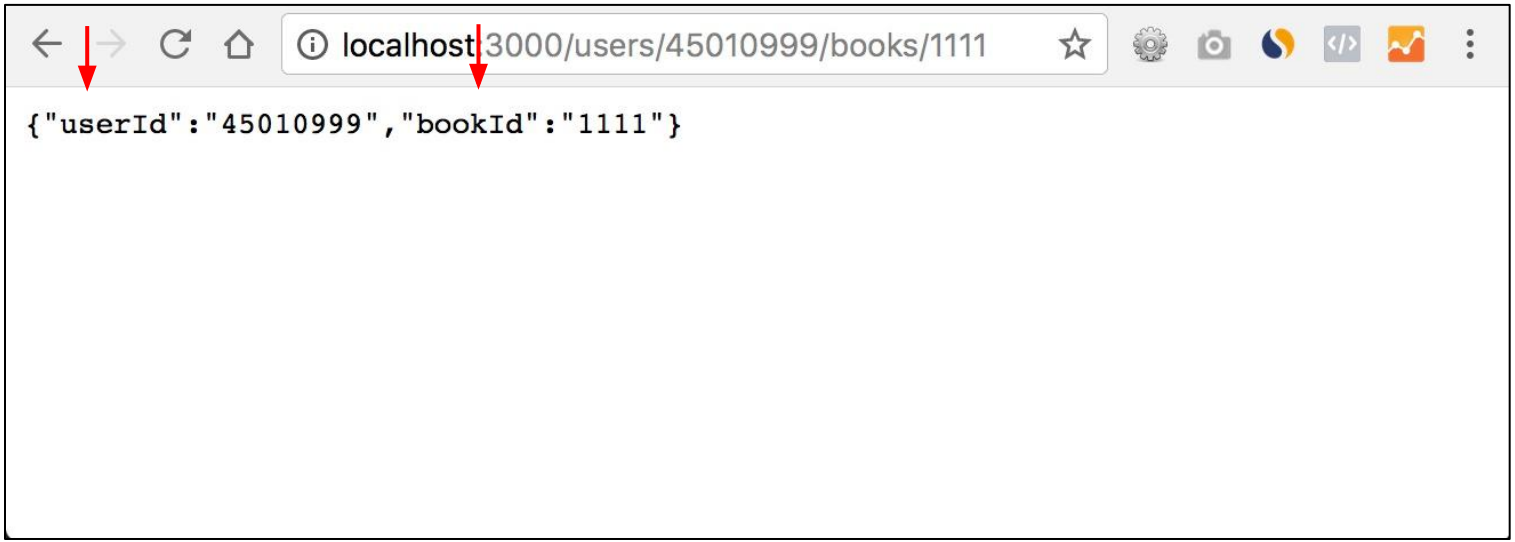
Group Route Methods with app.route()

```
app.route("/book")
    .get(function(req, res) {
        res.send("Get a random book");
    })
    .post(function(req, res) {
        res.send("Add a book");
    })
    .put(function(req, res) {
        res.send("Update the book");
    });
```

Route parameters

```
app.get("/users/:userId/books/:bookId", function(req, res) {
    res.send(req.params);
});
```

Output



Route Paths

Route Paths เป็นการกำหนด URI (Endpoints) ให้กับ Application นอกจากการกำหนดแบบตรงๆ แล้วสามารถกำหนดด้วย **String Pattern**, **Regular Expression** ได้เช่นกัน

```
app.get("/ab?cd", function(req, res) { // path| acd, abcd
    res.send("ab?cd");
});

app.get("/ab+cd", function(req, res) { // path| abcd, abbcd, abbbcd
    res.send("ab+cd");
});

app.get("/ab*cd", function(req, res) { // path| abcd, abxcd, ab123cd
    res.send("ab*cd");
});

app.get("/ab(cd)?e", function(req, res) { // path| /abe, /abcde
    res.send("ab(cd)?e");
});
```

Regular Expression

```
app.get(/a/, function(req, res) { // path| a
    res.send("/a/");
});

app.get(/.*fly$/, function(req, res) { // path| butterfly, dragonfly
    res.send("/.*fly$/");
});
```

Route handler

คือ **Callback Functions** ที่ถูกเรียกให้ทำงาน เมื่อ Client เรียกมาที่ Route Paths ที่กำหนดไว้

Route handler

```
app.get("/example/a", (req, res) => {
    res.send("Hello from A!");
});
```

Multiple Callback Functions

```
app.get("/example/b", (req, res, next) => {
        console.log("the response will be sent by the next function ..." );
        next();
    }, (req, res) => {
        res.send("Hello from B!");
    }
);
```

Array of Callback Functions

```
var cb0 = (req, res, next) => {
    console.log("CB0");
    next();
};
var cb1 = (req, res, next) => {
    console.log("CB1");
    next();
};
var cb2 = (req, res) => {
    res.send("Hello from C!");
};

app.get("/example/c", [cb0, cb1, cb2]);

app.get("/example/d", [cb0, cb1], (req, res, next) => {
        console.log("the response will be sent by the next function ...");
        next();
    }, (req, res) => {
        res.send("Hello from D!");
    }
);
```

express.Router

```
var express = require("express");
var router = express.Router();

router.get("/", (req, resp) => {
    resp.send("user router .....");
});

router.get("/:userId", (req, resp) => {
    resp.send(req.params);
});

module.exports = router;
```

Create & Export Module

```
var users = require('./users');
app.use('/users', users);
```

Import & Use Module

Serving static files in Express

Static files ประกอบด้วยไฟล์จำพวก วีดีโอ, รูปภาพ, JS, CSS และอื่นๆ ที่อยู่บน **Web Server**

```
http://localhost:3000/images/kitten.jpg
http://localhost:3000/css/style.css
http://localhost:3000/js/app.js
http://localhost:3000/images/bg.png
```

เพื่อให้ Express กำหนดที่อยู่บน Web Server เหล่านี้ขึ้น ทำได้โดยใช้คำสั่งด้านล่าง

Folder

```
app.use(express.static("public"));
```

กำหนด URL ให้ Static files

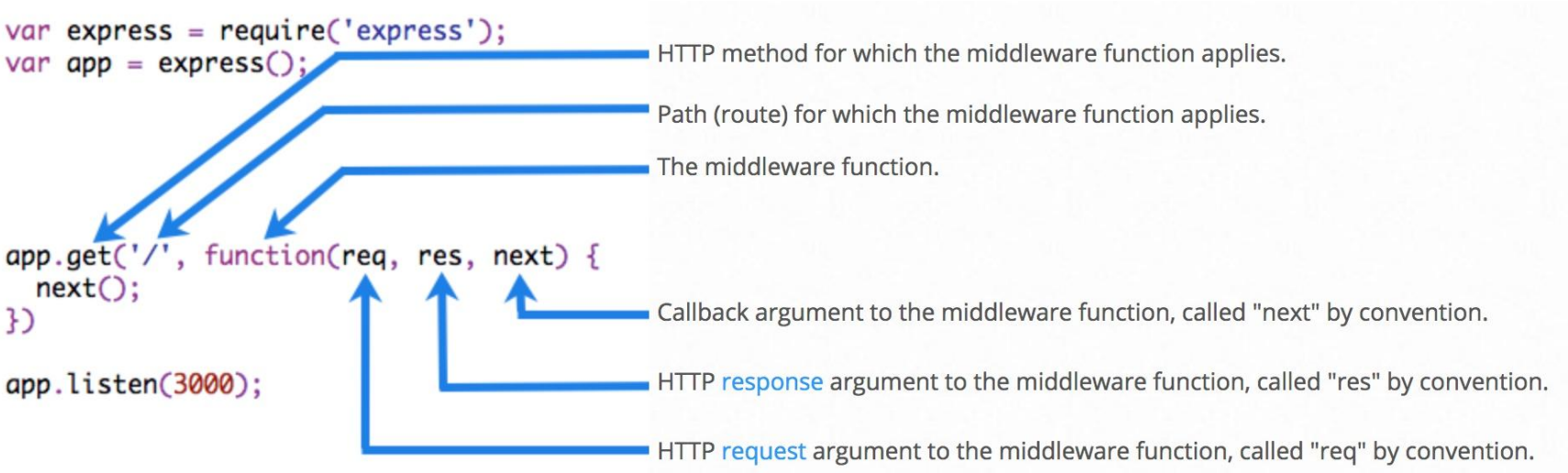
```
app.use("/asset", express.static("public"));
```

```
http://localhost:3000/asset/images/kitten.jpg
http://localhost:3000/asset/css/style.css
http://localhost:3000/asset/js/app.js
http://localhost:3000/asset/images/bg.png
```

Middleware

Middleware

Middleware คือ Functions ที่เข้าถึง req (HTTP Request), resp (HTTP Response) และเรียก Middleware ตัวต่อๆ ไปได้ผ่าน next()



Middleware Task

- เขียนโค้ดเพื่อทำงานเฉพาะทาง
  a. Log บันทึกข้อมูล
  b. ตรวจสอบสิทธิ์ผู้ใช้
  c. ตรวจสอบความถูกต้องของข้อมูลที่เข้ามา
- เปลี่ยนแปลงข้อมูล request , response
- จบการทำงาน request-response cycle
- เรียก Middleware ตัวถัดไปให้ทำงาน

Writing Middle

```
var express = require("express");
var app = express();

var myLogger = function(req, res, next) {
    console.log("myLogger| Time:", Date.now());
    next();
};

app.use(myLogger);

app.get("/", function(req, res) {
    res.send("Hello World!");
});

app.listen(3000, () => {
    console.log("Server listening on port 3000");
});
```

Middleware

Load Middleware

Configurable Middleware

```
module.exports = function(options) {
    return function(req, res, next) {
        console.log('my-middleware|', options);
        next();
    };
};
```

Middleware

```
var express = require("express");
var app = express();

var mw = require("./my-middleware.js");
app.use(mw({ option1: "1", option2: "2" }));

app.get("/", function(req, res) {
    var responseText = "Hello World!<br>";
    res.send(responseText);
});

app.listen(3000);
```

Load Middleware

Application Level Middleware

```
var express = require("express");
var app = express();

var requestTime = function(req, res, next) {
    req.requestTime = Date.now();
    next();
};

app.use(requestTime);

app.get("/", function(req, res) {
    var responseText = "Hello World!<br>";
    responseText += "<small>Requested at: " + req.requestTime + "</small>";
    res.send(responseText);
});

app.listen(3000);
```

Middleware

Load Middleware

Route Level Middleware

```
const express = require("express");
var path = require("path");
var app = express();
var router = express.Router();

router.use(function(req, res, next) {
    console.log("Time:", Date.now());
    next();
});
router.use("/user/:id", function(req, res, next) {
        console.log("Request URL:", req.originalUrl);
        next();
    }, function(req, res, next) {
        console.log("Request Type:", req.method);
        next();
    }
);
router.get("/user/:id", function(req, res, next) {
        res.send("user id : " + req.params.id);
});

app.use("/", router);
```

Load Middleware

Template Engines

Template Engines

Template Engines จะอ่านไฟล์ Template และแทนที่ตัวแปรต่างๆ ภายในไฟล์ Template เมื่อเรียกและส่งค่าตัวแปรเหล่านั่นเข้าไป ทำให้สะดวกในการออกแบบ HTML และลดการซ้ำซ้อน

Template Engines ที่นิยมประกอบด้วย **Pug**, **Mustache**, **EJS** เป็นต้น

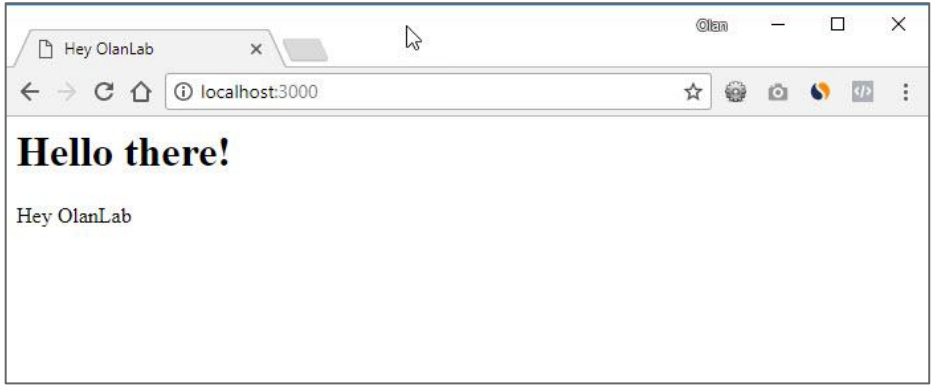
สามารถนำมาใช้กับ **Node.js + Express** ได้ทั้งหมด

PUG + Express

```
app.set("views", path.join(__dirname, "views"));
app.set("view engine", "pug");

app.get("/", (req, resp) => {
    resp.render("index", { title: "Hey OlanLab", message: "Hello there!" });
});
```

index.pug

```
html
 head
   title= title
 body
   h1= message
```



https://pugjs.org/api/getting-started.html

PUG

Doctype

```
doctype html
```

```
<!DOCTYPE html>
```

Plain Text

```
p This is plain old <em>text</em> content.
```

```
<p>This is plain old <em>text</em> content.</p
```

```
p
  | The pipe always goes at the beginning of
  | not counting indentation.
```

```
<p>The pipe always goes at the beginning of it
```

Plain Text

```
<html>

body
  p Indenting the body tag here would make no
  p HTML itself isn't whitespace-sensitive.

</html>
```

```
<html>

<body>
  <p>Indenting the body tag here would make n
  <p>HTML itself isn't whitespace-sensitive.</
</body>

</html>
```

Attributes

```
a(href='google.com') Google
|
|
a(class='button' href='google.com') Google
|
|
a(class='button', href='google.com') Google
```

```
<a href="google.com">Google</a>
<a class="button" href="google.com">Google</a>
<a class="button" href="google.com">Google</a>
```

```
- var authenticated = true
body(class=authenticated ? 'authed' : 'anon')
```

```
<body class="authed"></body>
```

Multiline Attributes

```
input(
  type='checkbox'
  name='agreement'
  checked
)
```

```
<input type="checkbox" name="agreement" checke
```

Class Attributes

```
- var classes = ['foo', 'bar', 'baz']
a(class=classes)
|
|
//- the class attribute may also be repeated
a.bang(class=classes class=['bing'])
```

```
<a class="foo bar baz"></a>
<a class="bang foo bar baz bing"></a>
```

Style Attributes

```
a(style={color: 'red', background: 'green'})
```

```
<a style="color:red;background:green;"></a>
```

Class Literal

```
a.button
```

```
<a class="button"></a>
```

```
.content
```

```
<div class="content"></div>
```

ID Literal

```
a#main-link
```

```
<a id="main-link"></a>
```

```
#content
```

```
<div id="content"></div>
```

Unbuffered Code

```
- for (var x = 0; x < 3; x++)
  li item
```

```
<li>item</li>
<li>item</li>
<li>item</li>
```

```
-
  var list = ["Uno", "Dos", "Tres",
          "Cuatro", "Cinco", "Seis"]
each item in list
  li= item
```

```
<li>Uno</li>
<li>Dos</li>
<li>Tres</li>
<li>Cuatro</li>
<li>Cinco</li>
<li>Seis</li>
```

Buffered Code

```
p
  = 'This code is <escaped>!'
```

```
<p>This code is &lt;escaped&gt;!</p>
```

```
p= 'This code is' + ' <escaped>!'
```

```
<p>This code is &lt;escaped&gt;!</p>
```

Interpolation

```
- var title = "On Dogs: Man's Best Friend";
- var author = "enlore";
- var theGreat = "<span>escape!</span>";

h1= title
p Written with love by #{author}
p This will be safe: #{theGreat}
```

```
<h1>On Dogs: Man's Best Friend</h1>
<p>Written with love by enlore</p>
<p>This will be safe: &lt;span&gt;escape!&lt;/
```

Conditionals

```
- var user = { description: 'foo bar baz' }
- var authorised = false
#user
  if user.description
    h2.green Description
    p.description= user.description
  else if authorised
    h2.blue Description
    p.description.
      User has no description,
      why not add one...
  else
    h2.red Description
    p.description User has no description
```

```
<div id="user">
  <h2 class="green">Description</h2>
  <p class="description">foo bar baz</p>
</div>
```

Iteration

each

```
ul
  each val in [1, 2, 3, 4, 5]
    li= val
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
</ul>
```

```
ul
  each val, index in ['zero', 'one', 'two']
    li= index + ': ' + val
```

```
<ul>
  <li>0: zero</li>
  <li>1: one</li>
  <li>2: two</li>
</ul>
```

```
ul
  each val, index in ['zero', 'one', 'two']
    li= index + ': ' + val
```

```
<ul>
  <li>0: zero</li>
  <li>1: one</li>
  <li>2: two</li>
</ul>
```

```
ul
  each val, index in {1:'one',2:'two',3:'three
    li= index + ': ' + val
```

```
<ul>
  <li>1: one</li>
  <li>2: two</li>
  <li>3: three</li>
</ul>
```

while

```
- var n = 0;
ul
  while n < 4
    li= n++
```

```
<ul>
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>3</li>
</ul>
```

Includes

```
//- index.pug
doctype html
html
  include includes/head.pug
  body
    h1 My Site
    p Welcome to my super lame site.
    include includes/foot.pug
```

```
//- includes/head.pug
head
  title My Site
  script(src='/javascripts/jquery.js')
  script(src='/javascripts/app.js')
```

```
//- includes/foot.pug
footer#footer
  p Copyright (c) foobar
```

```
<!DOCTYPE html>
<html>

<head>
  <title>My Site</title>
  <script src="/javascripts/jquery.js"></scri
  <script src="/javascripts/app.js"></script>
</head>

<body>
  <h1>My Site</h1>
  <p>Welcome to my super lame site.</p>
  <footer id="footer">
    <p>Copyright (c) foobar</p>
  </footer>
</body>

</html>
```

Includes Plain Text

```
//- index.pug
doctype html
html
  head
    style
      include style.css
  body
    h1 My Site
    p Welcome to my super lame site.
    script
      include script.js
```

```
/* style.css */
h1 {
  color: red;
}
```

```
// script.js
console.log('You are awesome');
```

```
<!DOCTYPE html>
<html>

<head>
  <style>
    /* style.css */

    h1 {
      color: red;
    }
  </style>
</head>

<body>
  <h1>My Site</h1>
  <p>Welcome to my super lame site.</p>
  <script>
    // script.js
    console.log('You are awesome');
  </script>
</body>

</html>
```

Error handling

Error handling

การจัดการข้อผิดพลาดที่เกิดขึ้นภายใน Application ภายใน Express สามารถทำได้ดังนี้

- สร้าง Middleware ที่จัดการ Error ไว้ด้านล่างสุด รับค่า err, req, res, next
- เมื่อ Middleware ใดมี Error ให้สร้าง Object ของ Error แล้วเรียก next(err)

Error : 404 Page Not Found

```
app.use(function(req, res, next) {
    var err = new Error("Page Not Found");
    err.status = 404;
    next(err);
});

app.use(function(err, req, res, next) {
    console.error(err.stack);
    res.status(err.status).send(err.message);
});
```

Express Application Generator

Express Application Generator

ติดตั้ง express-generator

```
npm install express-generator -g          # ติดตั้ง express-generator แบบ Global
```

คำสั่งต่างๆ ของ express-generator

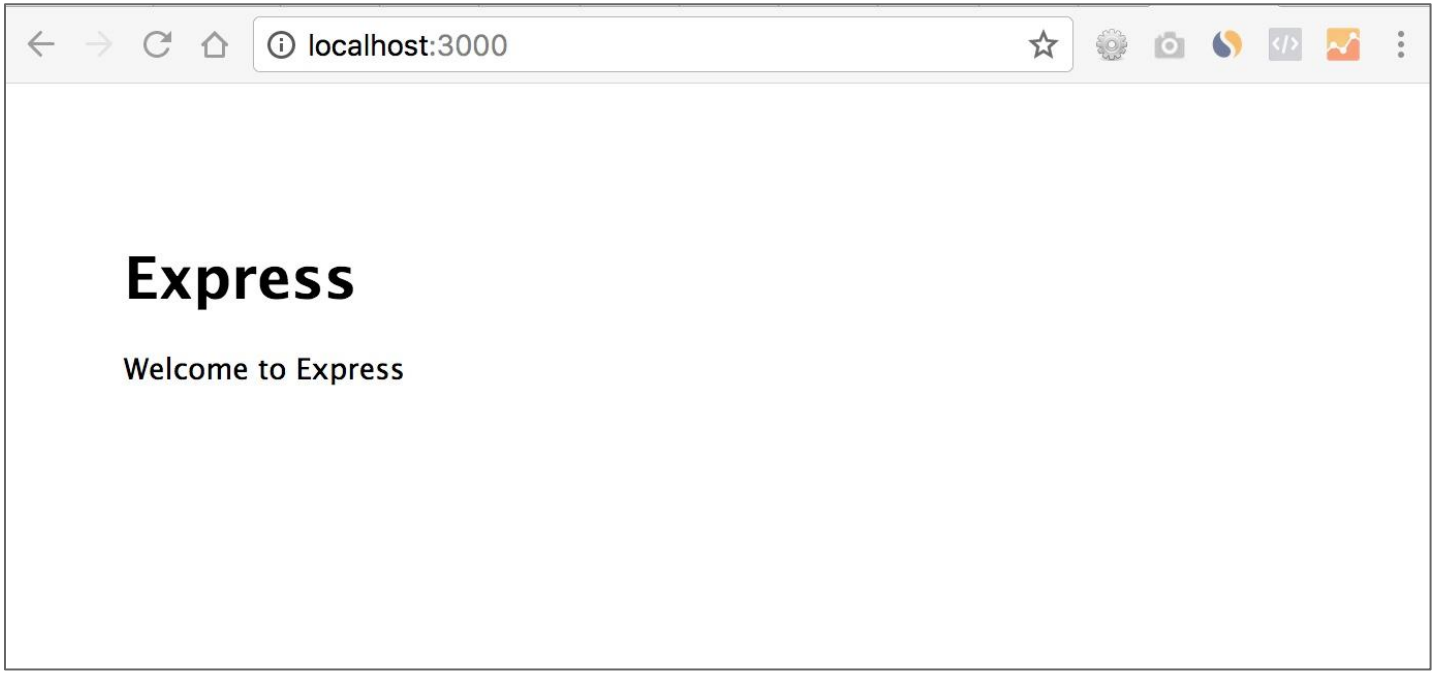
```
express -h                                    # ดูคำสั่งต่างๆ ของ express
```

```
Usage: express [options] [dir]


  Options:

        --version        output the version number
    -e, --ejs            add ejs engine support
        --pug            add pug engine support
        --hbs            add handlebars engine support
    -H, --hogan          add hogan.js engine support
    -v, --view <engine>  add view <engine> support
                         (dust|ejs|hbs|hjs|jade|pug|twig|vash) (defaults to jade)
    -c, --css <engine>   add stylesheet <engine> support
                         (less|stylus|compass|sass) (defaults to plain css)
        --git            add .gitignore
    -f, --force          force on non-empty directory
    -h, --help           output usage information
```

express-generator แต่คำสั่งใช้เพียง express

```
express --view=pug server        # สร้าง Project ชื่อ server ใช้ Template Engine (pug)
```

```
create : server
create : server/package.json
create : server/app.js
create : server/public
create : server/public/javascripts
create : server/public/images
create : server/routes
create : server/routes/index.js
create : server/routes/users.js
create : server/public/stylesheets
create : server/public/stylesheets/style.css
create : server/views
create : server/views/index.pug
create : server/views/layout.pug
create : server/views/error.pug
create : server/bin
create : server/bin/www
```

ติดตั้ง Modules ที่เกี่ยวข้อง และ สั่งให้ Web Server ทำงาน

```
cd server                    # เข้าไปข้างใน Folder : server
npm install                  # ติดตั้ง Module ภายใน package.json
npm start                    # เรียนคำสั่ง Start ใน package.json
```